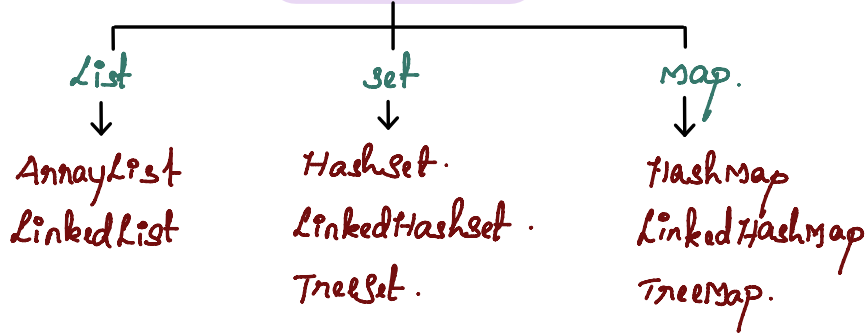


Collections.



List:-

ArrayList:-

It is dynamic memory Allocation.
Easy to update, insert, delete.

Syntax :-

className objName = new className(); (Normal class obj creation).

`ArrayList <Datatype> al = new ArrayList <>();`

Generics:

```
class Demo <T> {  
    T a;  
    T b;  
    println ( a + " " + b );  
}  
  
Demo <Integer> obj = new Demo <> ();  
obj. a = 20 ;
```

```
public class Main{  
    public static void main(String[] args) {  
        ArrayList <Integer> al = new ArrayList<>();  
        al.add(10);  
        al.add(6);  
        al.add(20);  
        al.add(30);  
        System.out.println(al);  
    }  
}
```

* List can store duplicate element.

```

public class Main{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        ArrayList <Integer> al = new ArrayList<>();
        for(int i=0;i<n;i++){
            int val = sc.nextInt();
            al.add(val);
        }
        System.out.println(al);
    }
}

```

```

public class Main{
    public static void main(String[] args) {
        ArrayList <Integer> al = new ArrayList<>();
        al.add(10);
        al.add(6);
        al.add(20);
        al.add(30);
        System.out.println(al);
        al.remove(1);
        System.out.println(al);
        System.out.println(al.get(0));
        al.add(2,68); //insert
        System.out.println(al);
        al.set(1,33); //update
        System.out.println(al);
        //al.clear(); //delete all
        System.out.println(al.size());
        System.out.println(al.contains(30));
        System.out.println(al.isEmpty());
    }
}

```

Collections.sort(al); //ascending order
 Collections.reverse(al); //descending order
 Collections.swap(al,0,2); //swap two values
 System.out.println(al);

Linked List

AddFirst - Add the value in first.

addLast - Add the value in last

getFirst - get the value in first

getLast - get the value in last

removeFirst - remove the value in first.

removeLast - remove the value in last.

peek - To print the first element.

poll - It print 1st element & delete that element.

```

public class Main{
    public static void main(String[] args) {
        LinkedList <Integer> al = new LinkedList<>();
        al.add(10);
        al.add(6);
        al.add(20);
        al.add(30);
        System.out.println(al);
        al.remove(1);
        System.out.println(al);
        System.out.println(al.get(0));
        al.add(2,68);//insert
        System.out.println(al);
        al.set(1,33);//update
        System.out.println(al);
        //al.clear();//delete all
        System.out.println(al.size());
        System.out.println(al.contains(30));
        System.out.println(al.isEmpty());
        //additional function for linkedList
        al.addFirst(11);
        al.addLast(55);
        System.out.println(al);
        System.out.println(al.getFirst());
        System.out.println(al.getLast());
        al.removeFirst();
        al.removeLast();
        System.out.println(al);
        System.out.println(al.peek());
        System.out.println(al.poll());
        System.out.println(al);
        LinkedList <Integer> al1 = new LinkedList<>();
        al1.addAll(al);
        System.out.println(al1);
    }
}

```

Set

- We can't access the value by their index.
- Duplicate elements are not allowed.

Hash set:-

- Element can be any order.
- Duplicate elements are not allowed only unique / key value.
- We can't access by index value.

```

public class Main {
    public static void main(String[] args) {
        HashSet <Integer> hs = new HashSet<>();
        hs.add(10);
        hs.add(6);
        hs.add(20);
        hs.add(30);
        hs.add(30);
        System.out.println(hs);
        hs.remove(6);
        System.out.println(hs);
        System.out.println(hs.size());
        System.out.println(hs.contains(20));
        System.out.println(hs.isEmpty());
        for(int val:hs){//for each loop
            System.out.print(val+" ");
        }
        System.out.println();
        //hs.clear();
        System.out.println(hs);
        HashSet <Integer> hs1 = new HashSet<>();
        hs1.addAll(hs);
        System.out.println(hs1);
    }
}

```

It doesn't follow any order.

Linked HashSet:

```

public class Main {
    public static void main(String[] args) {
        LinkedHashSet <Integer> hs = new LinkedHashSet <>();
        hs.add(10);
        hs.add(6);
        hs.add(20);
        hs.add(30);
        hs.add(30);
        System.out.println(hs);
        hs.remove(6);
        System.out.println(hs);
        System.out.println(hs.size());
        System.out.println(hs.contains(20));
        System.out.println(hs.isEmpty());
        for(int val:hs){//for each loop
            System.out.print(val+" ");
        }
        System.out.println();
        //hs.clear();
        System.out.println(hs);
        HashSet <Integer> hs1 = new HashSet<>();
        hs1.addAll(hs);
        System.out.println(hs1);
    }
}

```

Values are printed in the given order.

Tree set:

```
public class Main {
    public static void main(String[] args) {
        TreeSet <Integer> hs = new TreeSet <>();
        hs.add(10);
        hs.add(6);
        hs.add(20);
        hs.add(30);
        hs.add(30);
        System.out.println(hs);
        hs.remove(6);
        System.out.println(hs);
        System.out.println(hs.size());
        System.out.println(hs.contains(20));
        System.out.println(hs.isEmpty());
        for(int val:hs){//for each loop
            System.out.print(val+" ");
        }
        System.out.println();
        //hs.clear();
        System.out.println(hs);
        TreeSet <Integer> hs1 = new TreeSet <>();
        hs1.addAll(hs);
        System.out.println(hs1);
    }
}
```

values are print
in ascending order.

Map

- Map contains keys & value.
- keys should be unique.
- value can contains duplicate elements.

Syntax:-

`className < Datatype, Datatype> objName = new className <> ();`

↓ ↓
key value.

HashMap :-

```
public class Main {  
    public static void main(String[] args) {  
        HashMap<Integer, String> hm = new HashMap<>();  
        hm.put(111, "Pranav");  
        hm.put(201, "Gayathiri");  
        hm.put(402, "Bhavashri");  
        hm.put(507, "Gracy");  
        System.out.println(hm);  
        System.out.println(hm.get(111));  
    }  
}
```

```

        hm.putIfAbsent(333, "Mugil");
        System.out.println(hm);
        hm.putIfAbsent(507, "Gracy");
        System.out.println(hm);
        for(int key:hm.keySet()){
            System.out.print(key+" ");
        }
        System.out.println();
        for(String val:hm.values()){
            System.out.print(val+" ");
        }
        System.out.println();
        System.out.println(hm.getOrDefault(334, "Not found"));
    }
}

```

Linked HashMap:-

```

public static void main(String[] args) {
    LinkedHashMap<Integer, String> hm = new LinkedHashMap<>();
    hm.put(111, "Pranav");
    hm.put(201, "Gayathiri");
    hm.put(402, "Bhavashri");
    hm.put(507, "Gracy");
    System.out.println(hm);
    System.out.println(hm.get(111));
    hm.putIfAbsent(333, "Mugil");
    System.out.println(hm);
    hm.putIfAbsent(507, "Gracy");
    System.out.println(hm);
    for(int key:hm.keySet()){
        System.out.print(key+" ");
    }
    System.out.println();
    for(String val:hm.values()){
        System.out.print(val+" ");
    }
    System.out.println();
    System.out.println(hm.getOrDefault(334, "Not found"));
}

```

Tree Map:-

```

public class Main {
    public static void main(String[] args) {
        TreeMap<Integer, String> hm = new TreeMap<>();
        hm.put(111, "Pranav");
        hm.put(201, "Gayathiri");
        hm.put(402, "Bhavashri");
        hm.put(78, "Gracy");
        System.out.println(hm);
        System.out.println(hm.get(111));
        hm.putIfAbsent(333, "Mugil");
        System.out.println(hm);
        hm.putIfAbsent(78, "Gracy");
        System.out.println(hm);
        for(int key:hm.keySet()){
            System.out.print(key+" ");
        }
        System.out.println();
        for(String val:hm.values()){
            System.out.print(val+" ");
        }
        System.out.println();
        System.out.println(hm.getOrDefault(334, "Not found"));
    }
}

```

print element in ArrayList:-

```
public static void main(String[] args) {  
    ArrayList <Integer> al = new ArrayList<>();  
    Scanner sc = new Scanner(System.in);  
    int n = sc.nextInt();  
    for(int i=0;i<n;i++){  
        int x = sc.nextInt();  
        al.add(x);  
    }  
    System.out.print(al);  
}
```

ip:
5

40	20	20	10	50
----	----	----	----	----

op:-

40	20	20	10	50
----	----	----	----	----

Print max value:-

```
public static void main(String[] args) {  
    ArrayList <Integer> al = new ArrayList<>();  
    Scanner sc = new Scanner(System.in);  
    int n = sc.nextInt();  
    for(int i=0;i<n;i++){  
        int val=sc.nextInt();  
        al.add(val);  
    }  
    Collections.sort(al);  
    System.out.println(al.get(n-1));  
}
```

ip:
5

40	20	20	10	50
----	----	----	----	----

op:-

50 .

print min value:-

```
public static void main(String[] args) {  
    ArrayList <Integer> al = new ArrayList<>();  
    Scanner sc = new Scanner(System.in);  
    int n = sc.nextInt();  
    for(int i=0;i<n;i++){  
        int val=sc.nextInt();  
        al.add(val);  
    }  
    Collections.sort(al);  
    System.out.println(al.get(0));  
}
```

ip:
5

40	20	20	10	50
----	----	----	----	----

op:-

10

Reverse the Array :-

```
public static void main(String[] args) {  
    ArrayList <Integer> al = new ArrayList<>();  
    Scanner sc = new Scanner(System.in);  
    int n = sc.nextInt();  
    for(int i=0;i<n;i++){  
        int val=sc.nextInt();  
        al.add(val);  
    }  
    Collections.reverse(al);  
    System.out.println(al);  
}
```

ip:
5

10	20	20	40	50
----	----	----	----	----

op:-

50	40	20	20	10
----	----	----	----	----

Count even Number in ArrayList:-

```
public static void main(String[] args) {
    ArrayList <Integer> al = new ArrayList<>();
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    for(int i=0;i<n;i++){
        int val=sc.nextInt();
        al.add(val);
    }
    int count=0;
    for(int i=0;i<n;i++){
        if(al.get(i)%2==0){
            count++;
        }
    }
    System.out.println(count);
}
```

ip:
5

10	20	20	40	50
----	----	----	----	----

op:-
3.

Remove Duplicates :-

```
public static void main(String[] args) {
    ArrayList <Integer> al = new ArrayList<>();
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    for(int i=0;i<n;i++){
        int val=sc.nextInt();
        al.add(val);
    }
    LinkedHashSet <Integer> hs = new LinkedHashSet<>(al);
    ArrayList <Integer> al1 = new ArrayList<>(hs);
    System.out.print(al1);
}
```

ip:
5

10	20	20	40	50
----	----	----	----	----

op:

10	20	40	50
----	----	----	----

Swap 1st & last element:-

```
public static void main(String[] args) {
    ArrayList <Integer> al = new ArrayList<>();
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    for(int i=0;i<n;i++){
        int val=sc.nextInt();
        al.add(val);
    }
    Collections.swap(al,0,n-1);
    System.out.println(al);
}
```

ip:-

5

10	20	30	40	50
----	----	----	----	----

op:-

50	20	30	40	10
----	----	----	----	----

Second largest element:-

```
public static void main(String[] args) {
    ArrayList<Integer> al = new ArrayList<>();
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    for(int i=0; i<n; i++){
        int val=sc.nextInt();
        al.add(val);
    }
    Collections.sort(al);
    LinkedHashSet<Integer> hs = new LinkedHashSet<>(al);
    ArrayList<Integer> al1 = new ArrayList<>(hs);
    System.out.println(al1.get(al1.size() - 2));
}
```

ip:-

5

10	20	30	50	50
----	----	----	----	----

op:

30

Rotate one Array:-

```
public static void main(String[] args) {
    ArrayList<Integer> al = new ArrayList<>();
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    for(int i=0; i<n; i++){
        int val=sc.nextInt();
        al.add(val);
    }
    int val = al.get(n-1);
    al.remove(n-1);
    al.add(0, val);
    System.out.println(al);
}
```

ip:-

5

10	20	30	40	50
----	----	----	----	----

op:-

50	10	20	30	40
----	----	----	----	----

Frequency of an element:-

```
public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> al = new ArrayList<>();
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        for(int i=0; i<n; i++) {
            int val=sc.nextInt();
            al.add(val);
        }
        Collections.sort(al);
        int cnt = 1;
        for(int i=0; i<al.size()-1; i++) {
            if(al.get(i)==al.get(i+1)) {
                cnt++;
            }
            else {
                System.out.println(al.get(i)+" = "+cnt);
                cnt = 1;
            }
        }
    }
}
```

ip:

5

10	10	30	30	50
----	----	----	----	----

op:

10 = 2

30 = 2

50 = 1

String.

* String represented by " ".

* String is immutable. We can't change the original string.

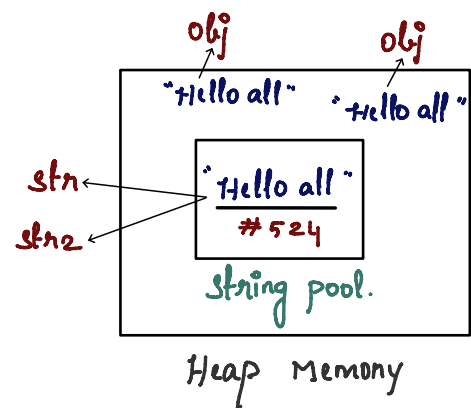
Declaration of String:-

Getting values from the user :-

```
public class Main {  
    public static void main(String[] args) {  
        Scanner sc= new Scanner(System.in);  
        String str1 = sc.next();  
        sc.nextLine();  
        String str2 = sc.nextLine();  
        System.out.println(str1);  
        System.out.println(str2);  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        String str = "Hello all";  
        String str1 = new String("Hello all");  
        System.out.println(str);  
        System.out.println(str1);  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        String str = "Hello all";  
        String str2 = "Hello all";  
        String str1 = new String("Hello all");  
        System.out.println(str);  
        System.out.println(str1);  
        System.out.println(str==str2);  
    }  
}
```



== → It checks only address not value

```
public class Main {  
    public static void main(String[] args) {  
        String str = "Hello all";  
        String str2 = "Hello all";  
        String str1 = new String("Hello all");  
        System.out.println(str);  
        System.out.println(str1);  
        System.out.println(str.equals(str1));  
    }  
}
```

equals() → It check the values not address.

`charAt(index)` → It take single character.

`length()` → It gives length of the string.

`substring(st idx, end idx)` → Part of the string.
 ↓ ↓
 included excluded.

`contains(str)` → check word present in the string.

`startsWith(str)` → Check if the string starts with that word.

`endsWith(str)` → Check if the string ends with that word.

`indexOf(str)` → It gives the first index of that word.

`lastIndexOf(str)` → It gives the last index of that word.

`replace(source, replaceWord)` → It replace the word with given word.

`replaceAll(Grop of val, replaceWord)` → It replace set of values.

`trim()` → It remove unwanted spaces in the string.

`toCharArray()` → Convert string into character array.

```
public class Main {
    public static void main(String[] args) {
        String str = " Hello Java, This is a Java Programming class";
        String str2 = "Hello all";
        String str1 = new String("Hello all");
        System.out.println(str);
        System.out.println(str1);
        System.out.println(str.equals(str1));
        System.out.println(str.charAt(2));
        System.out.println(str1.length());
        System.out.println(str1.substring(1,6));
        System.out.println(str1.substring(3));
        System.out.println(str.contains("This"));
        System.out.println(str.startsWith("Hello"));
        System.out.println(str.endsWith("Programming"));
        System.out.println(str.indexOf("Java"));
        System.out.println(str.lastIndexOf("Java"));
        System.out.println(str.replace("Java","C"));
        // It change vowels to '_'
        System.out.println(str.replaceAll("[AEIOUaeiou],"_"));
        // It change consonents to '_' (^ - Except vowels)
        System.out.println(str.replaceAll("[^AEIOUaeiou],"_"));
        System.out.println(str.trim());
        char arr[] = str1.toCharArray();
        for(char ch:arr){
            System.out.print(ch+" ");
        }
    }
}
```

Character Array to String:

```
public static void main(String[] args) {
    String str = " Hello Java, This is a Java Programming class";
    String str2 = "Hello all";
    String str1 = new String("Hello all");
    char arr[] = str1.toCharArray();
    for(char ch:arr){
        System.out.print(ch+" ");
    }
    System.out.println();
    //way1:
    String ans = "";
    for(char ch:arr){
        ans = ans+ch;
    }
    System.out.println(ans);
    //way2:
    String ans1 = new String(arr);
    System.out.println(ans1);
    //way3:
    String ans2 = String.valueOf(arr);
    System.out.println(ans2);
}
```

`Split()` → It split the string and stored in an array.

`Join()` → Merge the String array by given constraints.

```
public class Main {
    public static void main(String[] args) {
        String str = "Hello Java This is a Java Programming class";
        String ans[] = str.split(" ");
        for(String val:ans){
            System.out.println(val);
        }
        String ans1 = String.join("_",ans);
        System.out.println(ans1);
    }
}
```
